
Claude Code

在大型代码库中的工作方式

最佳实践与起步指南

来源:claude.com/blog • How Claude Code Works in Large Codebases

核心洞察

围绕模型搭建的**脚手架**——
*CLAUDE.md、Hooks、Skills、
Plugins、MCP*——比模型本身更能决
定 *Claude Code* 的表现。

— Anthropic 工程团队观察

模型 决定了能力上限

脚手架 决定了实际产出

大型代码库中,后者往往更具杠杆效应。

01 / 04

它如何导航代码库

没有索引,没有嵌入,只有活的代码

智能体搜索,而非 RAG 检索

传统 RAG 方式

- 依赖预先构建的向量索引
- 检索的可能是过期的代码片段
- 难以跨文件追踪引用关系
- 需要持续维护嵌入与索引

Claude Code 的方式

- 直接遍历文件系统
- 用 `grep` 精确定位符号
- 跨文件跟随引用链
- 面对的始终是活的代码

无需中心化索引——这正是它能跟上不断演化的代码库的原因。

02 / 04

Claude Code 脚手架

五个扩展点 + 两个加速器

五个扩展点全景



CLAUDE.md

每次会话自动加载的上下文文件



Hooks

关键时刻触发的脚本,用于自动化



Skills

按需加载的领域专长,避免上下文膨胀



Plugins

打包分发的组织级配置



MCP Servers

连接内部工具与数据源



加速器

LSP 提供符号级精度
Subagents 并行探索与编辑

CLAUDE.md — 每次会话的入口

分层布置,不要堆在一起

- 根目录的 `CLAUDE.md` 只放**普遍适用**的大局信息
- 子目录放本地约定:测试命令、lint 规则、模块惯例
- 更适合在子目录中初始化,而非仓库根
- 用 `.claudeignore` 排除生成代码与构建产物

保持精简、保持新鲜

CLAUDE.md 应当聚焦于"广泛适用"的内容——可复用的专长应该放到 Skills 里。

Hooks — 把规范变成自动化

在关键时刻触发的脚本,让人类约定**变成确定性行为**。

01 提交前格式化

统一代码风格,避免无意义的 diff

02 关键操作审计

对敏感命令记录与拦截

03 持续反馈回路

把测试结果、lint 错误回灌给会话

Hooks 把"团队希望发生的事"从口头约定升级为系统保障。

Skills — 按需加载的专长

解决"上下文膨胀"问题

- 不必要的专业知识不再塞进 CLAUDE.md
- 仅在任务类型匹配时被激活
- 把"如何写迁移脚本""如何打 release"封装成可复用单元
- 团队成员之间可以共享同一份专长

原则

可复用的专长 → **Skills**

而非永久驻留在每次会话里

Plugins — 让好配置可被分发

把 workflow 打包, 推广到全组织

- 统一的命令、Hooks、Skills 集合
- Day-one 即可使用, 无需个人摸索
- 避免好实践停留在"部落知识"
- 升级一处, 全员受益

一家最成功的部署案例: 在大规模推广前, 就已经"构建了一整套 *plugins* 与 MCP 服务器, 首日即可使用"。

— Anthropic 文章中的案例

MCP、LSP 与 Subagents



MCP Servers

把 Claude Code 接入内部工具、私有数据、票据系统、监控面板。

扩展"能做什么"



LSP 集成

用语言服务器做符号级查找,替代纯文本匹配,定位重命名与跨文件引用更精准。

提升"看得多准"



Subagents

并行探索与编辑,把噪声大的调研工作交给子代理,主上下文保持干净。

放大"能走多远"

注意:LSP 集成**不会自动发生**——需要主动配置。

03 / 04

三种配置模式

让代码库可导航 · 让配置不腐化 · 让组织有担当

模式一:让代码库可导航

结构

- 分层布置 `CLAUDE.md`:根目录概述,子目录细节
- 测试与 lint 命令按目录分别声明
- 目录结构特殊时,显式提供代码库地图

工具

- 用 `.claudeignore` 屏蔽生成代码与构建产物
- 部署 LSP 服务器,启用符号级搜索
- 常用的 MCP 服务器在 day-one 就配好

"保持 CLAUDE.md 精简且分层"——Anthropic 原文

模式二:配置随模型一起演进

3-6

个月

建议的回顾周期

模型在进步,你的规则不该原地踏步。

一条 *CLAUDE.md* 规则——“把每次重构拆成单文件改动”——可能曾帮过早期模型保持专注,却会阻止新模型做出协调一致的跨文件编辑。

每次重大模型发布后,审视并删除已不再需要的约束。

模式三:组织级的归属与治理

不指定负责人,配置就会碎片化。



设立 DRI

指定一位负责人或一支团队,统管中心化配置标准、模板与默认值。



保守起步

从一组**已审核的 Skills**、必备的 code review 流程、有限的初期访问开始。



跨职能工作组

早期就把工程、安全、治理代表拉到同一桌——而非事后救火。



随信心扩张

用 plugins 把已验证的工作流分发出去——避免好实践停留在"部落知识"。

04 / 04

陷阱与起步

先避开常见错误,再谈第一行配置

常见陷阱

⚠ 别这样做

- 把可复用专长塞进 `CLAUDE.md` (应放进 Skills)
- 什么都往 `CLAUDE.md` 里堆
- 以为 LSP 集成会自动发生
- 基础还没跑通就先搭 MCP 接入
- 只改了一个服务,却跑全量测试套件
- 好配置只在少数人手里,缺乏分发机制

✅ 这样替代

- 专长 → Skills;约定 → Hooks;广义上下文 → `CLAUDE.md`
- 每条规则都问:"这是不是普遍适用?"
- 把 LSP、子目录配置作为首日任务
- 先把"导航 + 测试 + lint"这三件事做对
- 用目录级 lint/test 命令限定影响面
- 用 plugins 把好实践打包传播

从哪里开始?

01 先写一份精简的 CLAUDE.md

只放普遍适用的内容,放进根目录;在子目录里再补本地约定。

02 把测试与 lint 命令按目录划清

加上 `.claudeignore`,让导航不被生成代码干扰。

03 部署 LSP,再考虑 MCP 与 Subagents

基础没跑通前,不要急着接外部系统。

04 指定 DRI,用 Plugins 分发好配置

保守起步、逐步扩张,每 3-6 个月回顾一次配置。

脚手架,而不是模型,决定你的 Claude Code 体验。